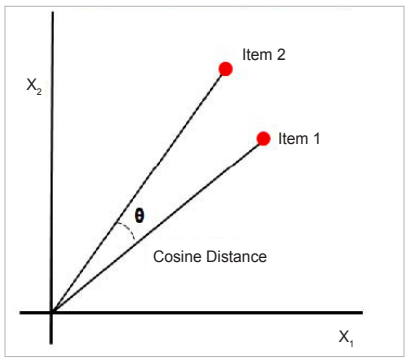


Figure 7: Cosine similarity

representations of the same text. But before we get into the specifics of word embedding, we need to consider the following question: Why do we need it?

Many machine learning algorithms and almost all deep learning architectures are unable to interpret strings or plain text in their raw form, as we all know. They need numbers as inputs to accomplish any task, including classification, regression, and so forth. Word embedding is nothing but the process of converting text data to numerical vectors. These vectors are then utilised to create a variety of machine learning models. In a sense, we could describe this as extracting features from text in order to create numerous natural language processing models.

We can transform text data into numerical vectors in a variety of ways. We'll learn about numerous word embedding strategies with examples in this post, as well as how to apply them in Python.

## Techniques for word embedding

The common and easy word embedding methods for extracting characteristics from text are:

- TF-IDF
- Bag of words
- Word2vec

### TF-IDF

A popular word embedding technique for extracting features from corpus or language is TF-IDF. It is a statistical way of finding how important a word is to a document over other documents.

TF stands for 'term frequency'. In TF, we assign a score to each word or symbol according to how frequently it appears. The frequency of a word is determined by the document's length. This means that a word appears more frequently in huge documents than in small or medium documents. To solve this problem, we can normalise the frequency of a word by dividing it by the length of the document (total number of words).

Using this method, we may create a sparse matrix with the frequency of each word.

```
TF=Number of times a term occurs in a document/total number of words in a document
```

### IDF

The full form of IDF is 'inverse document frequency'. Here, we are assigning a score value to a word, which explains how infrequent a word is across all documents. Infrequent words have higher IDF scores.

```
IDF = log base e (total number of documents/number of documents that have the term)
```

```
TF - IDF = TF * IDF
```

TF-IDF value is increased based on the frequency of the word in a document. This approach is mostly used for document/text classification, and also successfully used by search engines like Google as a ranking factor for content.

When compared to the words that are relevant to a document, some common words (like 'a', 'the', 'is') appear relatively frequently in a huge text corpus. These popular words convey very little information about the document's actual contents. If we fed the count data of these commonly occurring words directly to a classifier, the frequencies of rarer but more intriguing phrases would be overshadowed. So, ideally, what we want is to give lesser weightage to the common words occurring in almost all documents and give more importance to words that appear in a subset of documents.

The TF-IDF transform is frequently used to re-weigh count features into floating point values suitable for use by a classifier. This technique considers the occurrence of a word over the entire corpus, not just in a single document. Consider a business article — it will have more business-related terms like stock markets, prices, shares, and so on, than any other article, but terms like 'a', 'an', and 'the' will also appear frequently in it. As a result, this technique will punish certain high-frequency words.

Consider the table given below, which shows the total number of terms (tokens/words) in two papers.

| Document 1 | | Document 2 | |
|---|---|---|---|
| TERM | COUNT | TERM | COUNT |
| this | 1 | this | 1 |
| is | 1 | is | 2 |
| about | 2 | about | 1 |
| pandemic | 4 | technology | 1 |

Let's define a few terminologies associated with TF-IDF now.

```
Term Frequency (TF) = (Number of times term t appears in a document)/(Number of terms in the document)
So, TF (This, Document1) = 1/8
TF (This, Document2) =1/5
```

This denotes the word's contribution to the document, i.e., terms that are relevant to the document should appear frequently. For example, a document discussing a pandemic should use the word 'pandemic' frequently.

```
IDF = log (N/n), where, N is
```